Global Illumination with GPU Vertex Connection and Merging

João Monteiro
jna.monteiro@gmail.com

Instituto Superior Técnico, Lisboa, Portugal

October 2020

Abstract

There have been continuous developments in the field of computer generated graphics. In 2012, a new algorithm was presented, the vertex connection and merging, which consists in using different already existing algorithms to process a scene, to get the generated images with best quality among them. The main disadvantage of this approach, which is also present on each of the rendering algorithms, consists in the time it takes to create a realistic render. One of the possible approaches to reduce the time needed to process the information is the usage of a graphics processing unit, in order to take advantage of its multiple cores and parallelization capabilities. The rendering algorithms use, by nature, memory not in the most efficient way, with cache memory as an example. This is due to paths traced between camera and light sources having the possibility to be created in very distinct directions, even for adjacent pixels, which causes distant memory positions to be accessed, resulting in cache misses. On the other hand, CUDA applications are the most efficient when efficient memory access patterns are used. This thesis explores the conversion of an implementation of the vertex connection and merging algorithm, originally developed to be processed on the CPU, to be processed using NVIDIA GPUs. The goal of this conversion is to have images generated faster by the computer. The different types of memory in the GPU are used to optimize its performance, and a detailed analysis of the performance is done using the profiling tools provided by NVIDIA. Finally, this analysis explains why the obtained results are different from the expected, being the images generated faster using the original algorithm implementation.
Keywords: CUDA, GPU, VCM, parallelism, rendering

1. Introduction

Creating physically based renders has been an active subject of research since James Kajiya introduced the light transport equation, also known as rendering equation, in 1986 [9]. Until now, the most efficient way to generate these renders still takes a great amount of processing power, and is not normally done in real time. Physically based rendering takes into account physical equations, in order to simulate accurate lighting, where the law of conservation of energy is taken into account. Complex scenes typically contain multiple light sources and hundreds or thousands of objects, where the light bounces. All these properties make complex scenes very computationally intensive to be generated with enough accuracy and with reduced or no noise, and so, different rendering algorithms have appeared over the years. Vertex connection and merging (VCM), one of the state-of-the-art rendering algorithms presented in 2012, combines the results of different algorithms. The simulated light photons, generated by photon mapping, are connected to the camera using path tracing, and the renders can be processed faster, as less paths are traced, with the advantage of resulting in less noisy renders. Along with the evolution of rendering algorithms, the processing capabilities of central processing units (CPU) and graphics processing units (GPU) have also been improved through the years, allowing much more data to be processed than before. Applications that have an inherently parallel architecture and require a lot of processing power have been developed to be processed using GPUs, as these are specifically designed to process tasks in parallel. Since NVIDIA introduced CUDA in 2006 [1], and the Khronos Group introduced OpenCL in 2008 [6], the development of software to be processed on the GPU was made much easier, allowing developers to take advantage of the parallel processing capabilities of this hardware. As the rendering algorithms consist mainly in repeating the same tasks for each image pixel generated, GPU are perfectly suitable for processing them. The goal of this thesis is to implement VCM using CUDA, in order to be processed on NVIDIA GPUs and take advantage of their parallelism capabilities. The

source code basis for the algorithm implementation is SmallVCM, an application developed in C++ by the algorithm's author.

2. Background

2.1. Illumination

One of the most important elements of an image is illumination. Computer generated scenes have a concept of light, and its physical properties are simulated to achieve a very high degree of realism. Besides the properties of an object material, the light is also responsible for the final colors obtained in the scene. Just like in the real world, the light that illuminates an object may be in direct or indirect line of sight, being it direct or indirect illumination, respectively. In the case where an object is indirectly lit, this light is transmitted by other objects.

2.2. Multiple importance sampling

Samples resulting from different rendering algorithms can be combined, in order to get the advantages of them in different locations in a scene. Multiple importance sampling was presented by Veach in 1995 [12], and allows the combination of these different sampling techniques by calculating the weight that each is going to have in the final result. This is done by calculating the variance of each of the techniques used, and giving it a weight in the final combination. Without using MIS, when rendering a scene where multiple results get combined, there can be a high variance in the samples, and summing the different sampling results worsens the problem, leading to noise in the final render. Figure 1 shows the result of combining two rendering techniques using MIS.

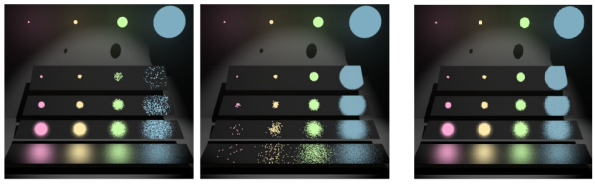


Figure 1: Sampling the light sources, sampling the BSDF and MIS (Source: Veach 1995 [12])

2.3. Ray tracing

In the ray tracing (RT) algorithm, presented by Turner Whitted in 1980 [13], a ray is shot from the camera towards the screen, hitting then the objects in the scene. This was the first algorithm that recursively traced additional rays when the initial ray hit an object that was not diffuse. These new additional rays can have different types, being reflection, refraction or shadow rays.

2.4. Path tracing

The path tracing (PT) algorithm, which was presented by Kajiya in 1986 [9], also renders an image by shooting a ray from the camera towards the screen, but the ray bounces from the objects until it reaches a light in the scene. When the ray intersects an object, the bidirectional scattering distribution function (BSDF) is taken into account.

2.5. Light tracing

The light tracing (LT) algorithm, which was presented by Arvo in 1986 [3], uses an approach that is the inverse of the one used in PT. In LT, the ray is cast from the light source, bouncing in the objects according to their BSDF. This algorithm allows the rendering of caustics, but the specular reflections seen by the camera cannot be calculated.

2.6. Bidirectional path tracing

In 1998, Veach presented a new algorithm called bidirectional path tracing [11] (BDPT). This algorithm uses both PT and LT, and after a predefined number of iterations, the resulting paths of each algorithm are connected. Both the PT and LT paths are initiated at the same time. These paths can then be connected by using a single ray between them, resulting in a single path, or using multiple rays connecting all the points from both paths, resulting in multiple paths.

2.7. Photon mapping

The photon mapping (PM) algorithm was presented in 1996 by Jensen [8], and like BDPT, allows the connection of different types of paths. However, in the case of PM, whenever the rays originated in the light source intersect an object, being these intersection points the photons, their properties are stored on a map. After the photon tracing, PT is used. When a ray originated in the camera is close enough to a photon intersection point, these are merged.

2.8. Progressive photon mapping

Progressive photon mapping (PPM) is an algorithm presented by Hachisuka in 2008 [7], and improves the PM by reducing the amount of memory used. In this algorithm, the first pass is a regular RT pass, identifying all the visible objects from the camera. The subsequent passes are all photon tracing passes. Whenever a photon is traced, the resulting intersection point is compared with the RT intersections. If the photon is close to one of the RT intersections, its contribution is added to the illumination value, and the properties of the photon intersection are not stored in memory.

2.9. Vertex connection and merging

Vertex connection and merging was presented in 2012, by Georgiev [5]. This algorithm uses both BDPT and a reimplementation of PM as a path sampling technique, and allows the merging of both techniques. As the original implementation of the

PM algorithm does not increase its render quality by increasing the amount of samples processed, VCM reformulation of PM allows new iterations to increase quality just like the other algorithms, and be dependent on the number of samples.

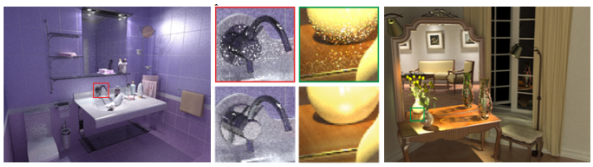


Figure 2: Merging of BDPT and PM using an heuristic (highlighted in red and green), and VCM with MIS (Source: Georgiev et al. 2012 [5])

To merge BDPT and PM, MIS is used. The render quality is then improved, giving all the benefits of the different algorithms. Small camera apertures and small lights can be rendered easily, while keeping the caustics generated by PM. Consequently, scenes with lots of different materials can now be rendered efficiently, without the noise that was generated with the previously used methods. Figure 2 shows the results of BDPT and PM being merged using VCM, eliminating most of the noise.

2.10. Light path tracing

The first stage of the algorithm is the tracing of light paths. Light paths have origin on each light source, and the resulting hit points, throughput and contribution for both connection and merging are stored. The stored data is called a light vertex. These contributions are later used to weight the contribution to the final color. Additionally, this stored data ensures that light paths are reused, in a similar way to how PM photons are used. When the light vertex is not purely specular and the path length limit is reached, the vertex is connected to the camera, contributing to the final color.

2.11. Camera path tracing

The second stage of the algorithm is the tracing of eye or camera paths. These paths are calculated like in BDPT algorithm. Similarly to light paths, each camera path also has a limited length, being also divided into sub paths. The end of each camera sub path is a camera or eye vertex. Unlike the light vertices, camera vertices are not stored in memory. However, the color contribution for these paths is calculated according to different conditions. If a path does not intersect any geometry, and if there is an environment light, the color contribution for this path is based on this light color. On the other hand, if a path maximum length is reached without any of the camera vertices being close to a light vertex, the color contribution is then sampled from a random light. Finally, if a camera vertex is close enough to a light vertex, both are merged. Otherwise, both are connected. In VCM, each light vertex is assigned to a path index, and the corresponding camera vertex is also defined using the same path index.

2.12. Vertex connection

The vertex connection stage takes place inside the camera path tracing. The path index is used to determine which light vertices should be connected to the camera vertex. This is done by getting the light vertices that match the current camera path. Then, every light vertex previously identified is connected to the current camera vertex, as long as the total length of the path is not greater than the maximum path length. The total length of the path takes into account the current camera path length, and the path length for each light vertex being connected. Finally, the camera vertex is also connected to a random light source. This stage is illustrated in figure 3.

2.13. Vertex merging

After the vertex connection stage, vertex merging takes place, which is illustrated in figure 3. Here, the light vertices are merged with camera vertices. This merge occurs when both vertices are within a specified radius, the acceptance radius.

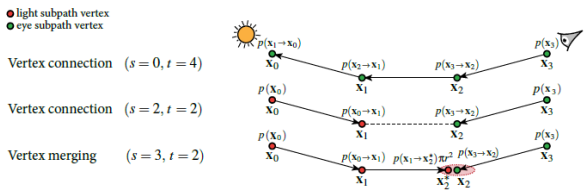


Figure 3: Camera and light paths with different lengths, and merging of camera and light vertices within the acceptance radius (Source: Georgiev et al. 2012 [5])

When a scene is rendered using multiple iterations, if the vertices are merged using the same radius between iterations, the final render may contain errors by being blurred. In order to minimize these errors, the radius is constantly decreased, in subsequent iterations, needing then the vertices to be closer to be merged. Initially the radius has a value $\mathbf{r_1}$, and in each iteration, the value is reduced proportionally to a constant α, as proposed by the author in equation (1).

$$r_i = r_1\sqrt{i^{\alpha-1}} \tag{1}$$

2.14. Algorithm overview

As discussed previously, VCM runs in two different main stages, being these the light path sampling and the camera path tracing. During the camera path sampling stage, vertex connection and merging

also take place. Finally, after each iteration, the acceptance radius is reduced. The pseudocode for the algorithm is listed in 1, based on the algorithm described by the author of VCM [4].

```
// Renders progressively by averaging the
    results from N independent iterations
function ProgressiveRender(r1, α, N, image
    )
  for i = 1 to N do
    iterationImage = BlackImage()
    r_i = r_1 √(i^(α−1))
    Render(r_i, iterationImage)
    image = (i−1)/i image + 1/i iterationImage
  end for
end function

// Renders a single image with a given
    maximum vertex merging radius
function Render(r)
  // Stage 1: Light path sampling
  for i = 1 to pixelCount do
    lightVertex = TraceLightRay(
        SamplePixel(i))
    while lightVertex is valid do
      if lightVertex is not specular
        StoreLightVertex(lightVertex)
        ConnectToCamera(lightVertex)
      end if
      lightVertex = ContinueRandomWalk(
          lightVertex)
    end while
  end for

  BuildRangeSearchStruct(lightVertices)

  // Stage 2: Camera path sampling and
      vertex connection and merging
  for i = 1 to pixelCount do
    cameraVertex = TraceCameraRay(
        SamplePixel(i))
    while cameraVertex is valid do
      // Unidirectional sampling (US)
      if cameraVertex is emissive then
        Accumulate(cameraVertex, US, r, i)
      end if
      // Vertex connection (VC)
      for lightVertex in lightPaths[i] ∪
          SampleLightPoint() do
        Accumulate(Connect(cameraVertex,
            lightVertex), VC, r, i)
      end for
      // Vertex merging (VM)
      for lightVertex in RangeSearch(
          cameraVertex, r) do
        Accumulate(Merge(cameraVertex,
            lightVertex), VM, r, i)
      end for
      cameraVertex = ContinueRandomWalk(
          cameraVertex)
    end while
  end for
end function

// Accumulates the pixel measurement
    estimate due to a given path
function Accumulate(path, technique, r, i)
  contrib = MeasurementContribution(path,
      technique, r)
  pdf = PDF(path, technique, r)
  weight = PowerHeuristic(path, technique,
      pdf)
  image[i] += weight * contrib / pdf
end function
```

Listing 1: VCM algorithm pseudocode

During the light path tracing, the first main stage introduced in 2.10, light paths are created from light sources. When the emitted ray intersects an object in the scene, the light vertex information is stored, and a new ray has a probability of being emitted from the intersection point. This process continues until the path reaches the maximum length, or until an intersection in the scene is not found. When each intersection occurs, the color contribution of the light vertices is also stored in the frame buffer, by connecting the camera to these vertices. Before tracing the camera paths, which is the second main stage, a range search data structure is created, which is later used to detect the light vertices that are closer to the camera vertices, in order to perform both vertex connection and merging operations. This detection is done using the radius that is defined for the current render iteration. The camera path tracing stage, introduced in 2.11, is then run for every pixel of the image. It starts by emitting a ray from the camera position, and detecting intersections, generating camera vertices. If there is an intersection with a purely emissive material, a light source, the color contribution is added to the frame buffer. All camera vertices are connected to all the light vertices assigned to the current path, connected to a random light, and if there are any light vertices inside the range defined by the iteration radius, these are also merged with the camera vertex. These contributions are then added to the frame buffer. The connection and merging stages are introduced in 2.12 and 2.13, respectively. After a camera vertex is created, a new sub path is created based on russian roulette probability, or if the path has not reached the maximum length yet.

2.15. SmallVCM

VCM has an experimental implementation, SmallVCM, created by its author [2]. Here, the different techniques used to render an image using the VCM algorithm are implemented and executed solely on the CPU, with multi threading support. When rendering an image, multiple renderers of the VCM algorithm are created and processed in parallel, using OpenMP. The number of renderers used is equal to the number of processors Each renderer has differences in the MIS weight calculation, related with the acceptance radius. After each iteration, the results from each renderer are stored in different

buffers and then combined, to get the final render. Each of these renderers is, in fact, rendering individual iterations. The implementation of VCM follows the approach described in the algorithm pseudocode listed in listing 1. Initially, light paths are traced, followed by camera paths. Light and camera vertices are then connected and merged in the camera path tracing stage, using a range search data structure, which is calculated between the light and camera path tracing stages.

2.16. Parallel processing using GPU
Due to the nature of the GPU hardware, which contains more transistors to process data, a high number of tasks can run in parallel. When dealing with graphics, these are processed using vertex or fragment shaders, among others. Before the appearance of NVIDIA CUDA in 2006, there was an era where the GPU started being used to process complex science and engineering problems [10]. However, in order to be able to access the hardware resources, developers needed to create very complex shaders and data, in order to process the latter as textures or as 3D objects. This era coincided with the appearance of DirectX 9. When GPU were being used as general purpose hardware, they were called general purpose GPU (GPGPU).

2.17. NVIDIA CUDA
In 2006, NVIDIA introduced a new revolutionary programming model, CUDA [1], which allows the development of software to be processed in the GPU, without having to write pixel shaders. NVIDIA introduced a set of extensions in the C language, allowing the developers to write code to be executed in the GPU without having to create the previous complex shaders and data. This programming model was introduced in the G80 series, and since this series the GPU come with a number of CUDA cores.

2.17.1 GPU Processors
The hardware present in a CUDA enabled GPU has some important differences when compared to the previous one. The GPU contains processing cores denominated by streaming multiprocessors (SM), which also contain a number of streaming processors (SP). SP are also denominated as CUDA cores.

2.18. GPU threads
The SM process threads in sets of 32, called warps. These are especially important when a warp is accessing global memory, which is a slow process, as a new warp starts its execution, reducing the time that the hardware is waiting for data to process.

2.19. GPU Memory
Unlike the host system RAM, a GPU has different types of memory available. There are two different groups of memory in a GPU, being one the device memory, where all data is accessible to all SM, and the SM memory itself, only accessible to each SM and corresponding threads. Global, local, constant and texture memory types are located in the device memory (off-chip memory), while the shared memory and registers are located on each SM (on-chip memory), being these the fastest memory types.

3. Implementation
In order to implement VCM using CUDA, SmallVCM is used as the code base, and changes are made on top of it, as mentioned previously.

3.1. CUDA threads
Each iteration processes all pixels of an image. As the number of threads is the same number of pixels, the total number of threads is then $W * H$ threads. For each pixel, each thread then processes all paths. The paths cannot be processed in parallel, as the values of subsequent paths depend on previous paths. For example, the Multiple Importance Sample (MIS) values from previous paths are divided by the BSDF values of the current path, not being possible to calculate these independently. Blocks are set to have a maximum of 256 threads. This ensures that there are enough warps ready to be started when others are idle, when data is being read from memory. With this number of threads, there are 8 warps per block. Finally, grids are set to have a number of blocks where the number of blocks multiplied by the threads per block match the number of pixels to process.

3.2. CUDA memory
During kernel execution, the objects that are accessed on every frame are copied from global memory to shared memory. Each SM has its own copy of the light objects, in order to reduce memory access collisions caused by threads from different SM. The scene object is also stored in the shared memory. The frame buffer is only stored in the global memory, as each pixel that is being processed can have the corresponding color contribution assigned to a completely different pixel in the resulting frame buffer. However, this is the slowest memory in the GPU, and in the worst case, a thread stores a contribution per calculated path.

3.3. CUDA kernels
Processing intensive tasks, which can also be implemented in parallel, are implemented using different kernels. This means that both vertex connection and vertex merging are fully implemented using separate kernels.

3.3.1 Kernel configurations
Before discussing the implementation of the different kernels, it is important to clarify that there are

three different kernel configurations, according to type of memory used, and when light vertices are connected to the camera. The main configuration, also identified as configuration 1, uses only global memory, not considering cache transactions, for all memory transactions that take place during the application lifetime. In configuration 2, the light vertices connection to camera is done in a dedicated kernel, after the light path tracing kernel is executed. Finally, configuration 3 adds shared memory to configuration 2, to store some static data that is read often, with lights as an example. Table 1 summarizes the configurations used.

Configuration	Global	Shared	Connection
1	Yes	No	No
2	Yes	No	Yes
3	Yes	Yes	Yes

Table 1: Kernel configurations according to memory and kernels used

3.3.2 Kernels implemented

The two main kernels that are implemented are *GenerateLightPaths* and *GenerateCameraPaths*. As the name implies, these are used to trace light and camera paths, respectively. Two additional kernels are used, *ResetIterations* and *GetFrameBufferData*, to set the GPU global memory to a valid state on each iteration, and to get the final frame buffer data, when all iterations are finished, respectively. These previously mentioned kernels are used in all configurations. Configurations 2 and 3 use two additional kernels to allow postponing the camera connection in the light path tracing kernel and the use of shared memory. These consist in a kernel to connect the light vertices to the camera, *ConnectLightPathToCamera*, and a kernel to reset the information if a light vertex should be connected to the camera, *ResetConnections*.

3.3.3 Kernel setup

Before running a kernel, all the required arrays are initialized in the host. Data that is required to run the kernels is also initialized and copied at this stage. For example, all the scene data is copied from the host to the GPU. This includes geometry, light and material data. If a kernel that runs multiple iterations at once is initialized, the number of iterations is also passed to the kernel as parameter.

3.4. Light path tracing

The light path tracing stage of the algorithm, introduced in section 2.10, has an overall implementation very similar to the implementation in SmallVCM.

3.4.1 Configuration 1

The first configuration consists in just replacing the outer most loop, which iterates over every pixel, in order to calculate the corresponding light paths. The path index is returned using the current block of threads and thread IDs running in the GPU.

3.4.2 Configuration 2

In this configuration, light vertices are not connected to the camera until all threads are finished in the light tracing stage. In order to be able to postpone the camera connection, all the data required must be stored in different arrays, in the global memory. This is to enable passing data between kernels, as the camera connection is processed in a different kernel. The kernel used to connect the light path with the camera uses a number of threads equal to the total number of sub paths for every pixel. If multiple iterations are being calculated at once, the number of iterations is also included. The total number of sub paths is then given by equation 2.

$$totalSubPaths = w * h * pathLength * iterations; \tag{2}$$

Using this approach results in a significant increase in GPU memory usage. This limits the maximum resolution that a render can have, as well as the iterations that can be processed on each kernel execution. For reference, the total required memory, in bytes, for a single sub path is 142 bytes.

3.4.3 Configuration 3

The third configuration uses shared memory to store the lights and part of the scene data, on top of configuration 2. The lights are part of the scene, in the original SmallVCM implementation, and in this configuration, the lights are copied to separate arrays in shared memory. However, the geometry data, which is also part of the scene, is not copied to the shared memory.

3.5. Camera path tracing

Following the same approach as in light path tracing stage, the camera path tracing stage of the algorithm, introduced in section 2.11, has an overall implementation very similar to the implementation in SmallVCM. It is at this stage that vertex connection and merging operations, introduced in sections 2.12 and 2.13 respectively, take place. This stage is implemented using two different configurations, 1 and 3.

3.5.1 Configuration 1

Similarly to light paths stage, the first configuration consists in just replacing the outer most loop. This loop iterates over every pixel, in order to calcu-

late the corresponding camera paths. As the loop is the same, the path index is returned using the current block of threads and thread IDs running in the GPU.

3.5.2 Configuration 3

In camera path calculation, for configuration 3, shared memory is again used to store the lights and the scene data. The data stored in shared memory is the same previously mentioned in light path tracing kernel, and light data is copied to shared memory by reusing the same function used in this kernel.

3.6. Hash grid

Between the light and camera path tracing stages, the hash grid is created based on the light paths data. The calculation of this data structure is performed by the CPU. Implementing the code similarly to the CPU implementation results in a very slow calculation, slowing down the whole rendering of the scene. Due to this reason, the light vertices data required for calculations is copied from GPU to host memory. After doing the hash grid calculation, the result is then copied to the GPU, to be used in the vertex merging stage.

3.7. Frame buffer

Color data for the final render is stored in the frame buffer. To improve light and camera path tracing kernel performance, each iteration has its own frame buffer. This ensures that when color data is being written, the number of collisions is reduced as much as possible. However, collisions can still occur for a given iteration, as different paths can write color data to the same location. The temporary color data is copied to the main frame buffer after calculating the contributions for all light and camera vertices, and after vertex connection and merging is performed.

3.8. Iteration reset

In order to have an iteration in a clean state, it is enough to reset the iteration frame buffer and the array with the path lengths for the light path tracing stage. All the other data is generated per iteration, without any intermediate memory that requires being reset.

3.9. Kernel configurations summary

CUDA VCM can be executed using 3 different configurations, where each configuration changes the types of memory used, and postpones the connection of light path vertices to camera. These configurations are summarized in table 1.

4. Evaluation methodology

When implementing a new solution or modifying an existing one, the results need to be evaluated, to check if it conforms to the specifications and if the objectives are achieved. In this case, the proposed solution consists in optimizing performance, while keeping the resulting images as close as possible to the original implementation. This means that the verification and validation is done by comparing both hardware performance, and generated images.

4.1. Scenes

In order to compare SmallVCM with CUDA VCM, we need to make sure that the conditions are similar. This means that the scenes to be tested, including the light conditions, must be the same. Otherwise, the render results are different, invalidating a valid comparison. In this case, the scenes are hard coded in the SmallVCM implementation, and the same scenes are used in CUDA VCM. In section 5, one scene consisting of a Cornell Box is used. This scene has an area light, and a large sphere with a mirror material, with reflectance equal to 1, is the single object present in the scene. The walls and ceiling of the box use diffuse materials, and the floor uses a glossy material.

4.1.1 Reference images

All generated images, including the ones generated by SmallVCM, are compared against reference images for the corresponding scenes. These images are generated using SmallVCM, with a total of 10000 render iterations each.

4.1.2 Iterations per scene

All comparisons are done against scenes rendered with the same amount of samples. Instead of checking how many iterations can be performed within a defined time interval, the number of iterations to be rendered are passed as an argument. This ensures that the image comparison is performed between images that should be equal, or with as less as differences as possible.

All scenes are rendered with the same amount of iterations, and each scene is rendered multiple times, each with a different number of iterations. The target iterations are 1, 5, 10, 20, 50 and 100. All there renders are considered for render time analysis. However, for image comparison purposes, only the renders with 100 iterations are considered.

4.1.3 Light and camera path length

The paths generated in both light and camera path tracing all have the same maximum length, and thus, the same number of sub paths or vertices. The length of these is set to 10, in all scenes.

4.2. Hardware

Final rendering tests are performed on up to date and high end hardware, providing accurate results

for the currently available technology. The specifications of the machine where CUDA VCM is executed are a AMD Ryzen 7 3700X 8-Core CPU, with a base clock frequency of 3.6GHz and 16GB of DDR4 RAM, and an NVIDIA GeForce RTX 2080 GPU, with 8GB of GDDR6.

4.3. Image comparison

Besides the performance comparison, the final render results also must be compared. The metrics used to compare images in the scope of this work are absolute error count (AE), peak signal to noise ratio (PSNR), root mean squared error (RMSE) and structure similarity index metric (SSIM).

5. Results evaluation

The GPU implementation of VCM is considered to be better than SmallVCM if the quality is similar or better, and if the processing time is smaller, for the same number of iterations. The image quality metrics are calculated according to the metrics described in section 4.3. The configurations can be found in table 1. As a reminder, the reference image is generated using SmallVCM, with a total of 10000 iterations. In the render results comparison, this is the figure identified as *reference*.

One very important remark about the results in this section is that the results from the GeForce RTX 2080 are taken from the application running with debug symbols enabled, and so, the results presented do not match the best case for CUDA VCM running on this hardware.

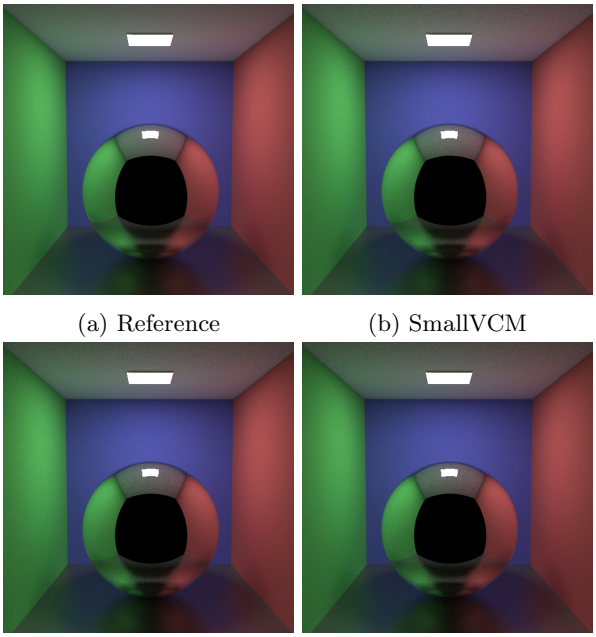
5.1. Scene evaluation

The scene consists of a single large sphere, with a mirror material applied to it, and a single area light, in the ceiling. The environment color is black, and is reflected in the front of the sphere. The render results are shown in figure 4, with the reference image on the left.

Rendering the scene using configuration 2 introduces some visible artifacts in some of the renders, depending on the number of iterations. The same applies to rendering the scene using configuration 3, where shared memory is used to store frequently accessed data. These artifacts are visible in figure 5.

Figure 6 shows the render times for the scene rendered with all target iterations, using a logarithmic scale.

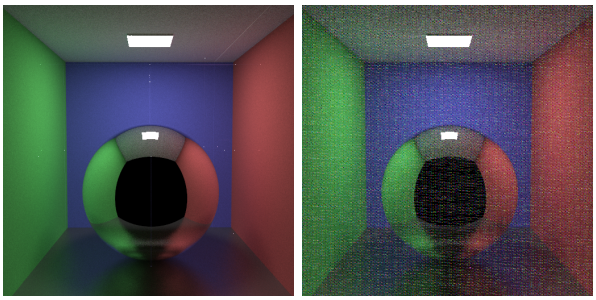
Render times for the scene with a target of 100 iterations are listed in table 2. Table 3 lists the differences between the render results for a target of 100 iterations and the reference render, with 10000 samples, for AE and PSNR metrics, and table 4 lists the differences for RMSE and SSIM metrics. For render time, the performance of both GPUs is significantly worse than the CPU, and the difference is



(a) Reference (b) SmallVCM

(c) Configuration 1 (d) Configuration 3

Figure 4: Scene 1 rendered with 100 frames, with image 4a as reference render



(a) Configuration 2 (100 i) (b) Configuration 3 (10 i)

Figure 5: Render artifacts in scene 1 for deferred camera connection and shared memory configuration

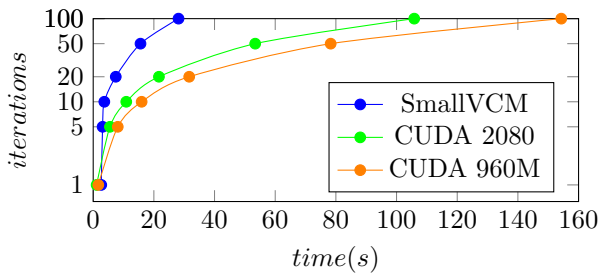


Figure 6: Render time for all iterations in scene 1

Configuration	Duration (s)
SmallVCM	28,148
Configuration 1	105,830
Configuration 3	106,326

Table 2: Render time for scene 1

Configuration	AE	PSNR (dB)
SmallVCM	241798	39,306
CUDA VCM	244044	36,252
Shared memory	246213	33,169

Table 3: Image comparison metrics for scene 1

proportional to the number of iterations. Deferring the camera connection and using the shared memory to store frequently accessed data does not have any significant performance impact, being slightly worse when shared memory is used. The quality of the resulting render for 100 iterations, using configuration 1, is the closest to the reference render. As previously seen in figure 5, the renders using kernel configurations 2 and 3 introduce artifacts in the renders, reducing the overall quality according to the metrics results.

5.2. Final considerations

Besides the much higher core count on a CUDA GPU, the code must be implemented using efficient data access patterns. Otherwise, the performance achieved can be significantly lower compared to a CPU, even when the number of threads is theoretically the maximum for the GPU, and faster memory types are used. In the case of rendering algorithms, it is also very difficult to have code optimized to all scene configurations, and render times can be significant different for scenes with similar object shapes, but with different materials or light configurations. Contrary to what is expected, all render times are significantly worse in the CUDA VCM implementation, when executed in NVIDIA GeForce RTX 2080. This is due to the application running with GPU debug information enabled, as CUDA compiler would crash otherwise. When the GPU debug information is enabled, the application runs

Configuration	RMSE (0-1)	SSIM
SmallVCM	2,762 (0,011)	0,910
Configuration 1	3,926 (0,015)	0,905
Configuration 3	5,599 (0,022)	0,905

Table 4: Image comparison metrics for scene 1

significantly slower due to all code optimizations being disabled. Some CUDA sample projects were also executed in NVIDIA GeForce RTX 2080, with both the GPU debug information flag enabled and disabled. The difference factor between executing the samples with the flag -G disabled and enabled is greater than 10. According to these results, the execution of CUDA VCM should theoretically be faster in the GeForce RTX 2080 GPU, compared to the CPU.

6. Conclusions

Fine tuning a CUDA application is a process that requires a deep understanding of how both the software, but most importantly, GPU hardware work. With each new architecture released by NVIDIA, memory access patterns evolve, and new hardware capabilities are added to the GPUs. Data optimization patterns should be used, in order to get the best performance out of the different types of memory available in the GPU, as the number of CUDA cores is not enough to increase performance, as it happens with faster CPUs. That being said, traditional software development techniques cannot be used, where trusting the compiler to optimize the software is often enough. In a CUDA application, the developer is responsible for the most critical optimizations, that may be the difference between a very fast or very slow application. Rendering algorithms have the particularity of the data in memory being accessed with high divergence, coupled with the infinite combinations of materials and objects that can be part of a scene. This high divergence is the result of rays being cast from the camera or from the light sources having a high probability of bouncing in very different directions.

6.1. Achievements

The main goal of this thesis, which consisted in having an implementation of the VCM algorithm to be executed faster on CUDA GPUs, when compared to an implementation to be executed on CPUs, was not achieved. Nonetheless, the overall work is proven to have been a very rich experience about parallel processing using CUDA, and its relation with rendering algorithms, in this case, by implementing VCM.

6.2. Code structure and hardware usage

Efficient code in a kernel is directly related with the amount of instructions that need to be executed, and hardware resources used in the GPU. Kernels must then consist of a good balance between number of instructions present in the code, which translates in number of registers used, number of threads, and memory allocated, both defining the number of warps that can be executed simultaneously in a SM. In the scope of CUDA VCM, there are also multiple

conditions that stop light and camera paths being traced, resulting in idle threads, which is then reflected in the overall performance of the GPU.

6.3. Memory usage

The best types of memory to be used should be decided based on how data is stored in memory, and how it is accessed. For rendering algorithms, it is not an easy task to optimize an application to make good use of memory. This thesis focused mostly in the usage of global memory, with some data being stored in shared memory. However, for future work, shared memory can be further explored, as well as constant memory.

6.4. Future Work

In this section, all the unfinished work is summarized, and any future work that is not discussed yet is added, where the current CUDA implementation can be used as a starting point.

6.4.1 Render a scene using smaller portions

The intrinsic divergence present in rendering algorithms is caused by rays bouncing in different directions, even for the same path, both for camera and light sub paths. Using a combination of rendering smaller portions of a scene, together with storing the data for these portions in adjacent memory positions, can be an efficient method to reduce render times.

6.4.2 Shared memory

Shared memory was implemented only for storing part of the scene properties, and the light sources. Geometry is a very good candidate to be stored in shared memory, as it is used for each sub path, for each iteration. Geometry is static during the whole scene rendering, and is used by all SM.

6.4.3 Hash grid calculation using GPU

The hash grid used to detect vertices that should be merged is implemented in the CPU. This means that there are memory transactions to copy light vertices related data to the system RAM, hash grid is calculated, and the result is then copied to GPU. The hash grid is implemented in the CPU due to the exclusive prefix sum (or scan), that when implemented in the GPU was slower than calculating it in the CPU.

6.4.4 Support for non square images

The current implementation only allows square images to be render. Due to this limitations, different aspect ratios cannot be rendered.

Acknowledgements

I would like to thank my thesis supervisor for all the help and support provided since the beginning of this thesis, and Instituto Superior Técnico for the high quality education I was provided.

References

[1] Nvidia cuda zone website. `https://developer.nvidia.com/cuda-zone`.

[2] Smallvcm website. `http://www.smallvcm.com/`.

[3] J. Arvo. Backward ray tracing. In *ACM SIGGRAPH '86 Course Notes - Developments in Ray Tracing*, pages 259–263, 1986.

[4] I. Georgiev. *Path Sampling Techniques for Efficient Light Transport Simulation*. PhD thesis, Saarland University, Saarbrücken, Germany, 2015.

[5] I. Georgiev, J. Křivánek, T. Davidovič, and P. Slusallek. Light transport simulation with vertex connection and merging. *ACM Trans. Graph.*, 31(6), 2012.

[6] K. Group. The khronos group releases opencl 1.0 specification. `https://www.khronos.org/news/press/the_khronos_group_releases_opencl_1.0_specification`, 2008.

[7] T. Hachisuka, S. Ogaki, and H. W. Jensen. Progressive photon mapping. *ACM Trans. Graph.*, 27(5), 2008.

[8] H. W. Jensen. Global illumination using photon maps. In *Proceedings of the Eurographics Workshop on Rendering Techniques '96*, page 21–30, 1996.

[9] J. T. Kajiya. The rendering equation. *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '86)*, 20(4):143–150, 1986.

[10] D. B. Kirk and W.-m. W. Hwu. *Programming Massively Parallel Processors, 2nd edition, A Hands-on Approach*. Morgan Kaufmann, 2012.

[11] E. Veach. *Robust Monte Carlo Methods for Light Transport Simulation*. PhD thesis, Stanford University, Stanford, CA, USA, 1998.

[12] E. Veach and L. J. Guibas. Optimally combining sampling techniques for monte carlo rendering. In *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '95, page 419–428, 1995.

[13] T. Whitted. An improved illumination model for shaded display. *Commun. ACM*, 23(6):343–349, 1980.